Paket Desain Arsitektur

Aplikasi Biro Umum dan Rumah Tangga · Kementerian Ketenagakerjaan

Empat layanan: Pemeliharaan, Pengadaan, Kendaraan Dinas, dan Ruang Rapat

Pemilik: Michael · 5 November 2026

Bagian	Isi	Tahap
1	Ringkasan kebutuhan: fitur, target kualitas, pengguna, scope, asumsi	Tahap 1
2	Catatan keputusan arsitektur (6 keputusan, beserta trade-off)	Tahap 2
3	Diagram C4: konteks, container, komponen, dan kode (6 diagram)	Tahap 3
4	Data (dua ERD) dan integrasi	Tahap 3
5	Akses dan risiko (tabel akses, 3 catatan risiko, kotak tempat kontrol)	Tahap 4

Cara membaca paket

- Kode **F1–F8** (fitur) dan **N1–N6** (target kualitas) di bagian 1 dipakai untuk merujuk kebutuhan di bagian lain. Kode **K1–K6** adalah keputusan, **R1–R3** adalah risiko. Halaman diagram C4 berukuran A3 landscape agar teksnya terbaca.
- Seluruh isi disusun dari kode dan skema database aplikasi yang sedang berjalan, bukan rancangan baru. Bagian yang berupa asumsi atau belum diuji ditandai tegas.
- Berkas pendamping: **Michael_Arsitektur.drawio** (bisa diedit; 9 halaman: Alur Booking, C4 Level 1–4 sebanyak 6 halaman, ERD Layanan, ERD Ruang Rapat). Diagram C4 yang sama juga ada terpisah di **Michael_C4.pdf**.

Bagian 1 · Ringkasan kebutuhan

Tahap 1 · F = fitur (functional), N = target kualitas (non-functional).

Fitur utama

Kode	Jenis	Kebutuhan
F1	Functional	Pegawai mengajukan permintaan **Pemeliharaan** (judul, lokasi, kategori, dokumen permohonan) dan **Pengadaan** (nama barang/jasa, metode, nilai HPS). Karyawan yang mengajukan otomatis menjadi PIC atas permintaannya.
F2	Functional	PIC, Kabag, dan Admin memproses tiap permintaan lewat tiga tahap berstatus pending → on_progress → selesai, dengan dokumen terlampir (mis. BoQ dan HPS, pembayaran GUP/TUP/LS beserta invoice, dokumentasi dan BAST). PIC hanya untuk data buatannya sendiri.
F3	Functional	Kabag dan Admin mengelola **Kendaraan Dinas**: nomor polisi, BPKB, STNK, pajak, foto, serta riwayat service dan invoice; semua pegawai dapat melihat. Dashboard menandai pajak yang jatuh tempo dalam 14 hari.
F4	Functional	Tamu masuk dengan akun Google di kiosk lalu mengajukan **booking ruang rapat**: ruangan, rentang atau beberapa tanggal pilihan, jam, nomor surat, file surat, dan captcha.
F5	Functional	Siapa pun melihat jadwal ruang rapat tanpa login. Sistem menolak booking yang bentrok (ruangan, tanggal, dan jam beririsan) dan ruangan yang sedang maintenance.
F6	Functional	Tamu mengedit atau membatalkan booking miliknya selama belum berjalan. Pegawai mengelola semua booking: membuat, mengedit, dan membatalkan seluruh booking atau tanggal tertentu beserta alasannya.
F7	Functional	Tamu menerima email konfirmasi (ajukan, edit, batal), email pembatalan oleh Biro Umum, dan pengingat H-1 serta 2 jam sebelum mulai.
F8	Functional	Kabag dan Admin mengelola akun, Mode Maintenance per menu, data ruangan (foto dan fasilitas), dan melihat riwayat aktivitas.

Target kualitas (bisa diuji)

Kode	Jenis	Target dan cara uji
N1	Performance	95% request daftar dan jadwal (mis. `GET /api/ruang-rapat/public-schedule`, daftar Pemeliharaan) selesai ≤ 2 detik dengan 1.000 pengguna aktif bersamaan di lingkungan uji (database terpisah berisi data skala produksi). **Target ini asumsi.** Uji beban 4 Nov 2026 pada kode lama menunjukkan lag di sekitar 200 pengguna aktif; optimasinya (K5) sudah diterapkan tetapi belum diukur ulang terhadap target ini.
N2	Security	Akun tamu A ditolak (404) saat mengedit atau membatalkan booking tamu B, dan booking B tidak berubah. Uji: dua token tamu.
N3	Security	Karyawan ditolak (403) saat mengedit atau menghapus data Pemeliharaan/Pengadaan; PIC hanya bisa data yang ia buat sendiri. Uji: token karyawan, PIC A, PIC B.
N4	Konsistensi	Dua pengajuan bersamaan untuk ruangan dan jam yang beririsan: tepat satu berhasil, yang lain ditolak 409 BENTROK. Uji: dua request paralel ke ruangan yang sama.
N5	Reliability	Gangguan SMTP tidak menggagalkan simpan atau batal booking: respons tetap sukses dan kegagalan tercatat di log. Uji: isi SMTP_PASS dengan nilai salah lalu ajukan booking.
N6	Auditability	Tambah, ubah, hapus, dan batal pada Pemeliharaan, Pengadaan, Kendaraan, Ruang Rapat, ruangan, dan akun tercatat di Riwayat Aktivitas beserta aktor dan nilai sebelum/sesudah. Uji: ubah satu data lalu periksa Riwayat. **Belum mencakup perubahan Mode Maintenance** (tidak dicatat).

Pengguna

Pengguna	Cara masuk	Peran di sistem
Tamu	Login Google di kiosk (sesi 7 hari)	Mengajukan, mengedit, dan membatalkan booking ruang rapat miliknya.
Pegawai (Karyawan, PIC)	Akun aplikasi (email dan kata sandi)	Melihat data semua layanan, mengajukan Pemeliharaan/Pengadaan, dan mengelola booking ruang rapat. PIC memproses tahapan data buatannya sendiri.
Kabag / Admin	Akun aplikasi	Seperti pegawai, ditambah kendaraan, akun, Mode Maintenance, riwayat aktivitas, dan data ruangan.

Batas scope

- **Di dalam:** empat layanan (Pemeliharaan, Pengadaan, Kendaraan Dinas, Ruang Rapat) beserta login tamu Google, email booking, akun, Mode Maintenance, dan riwayat aktivitas.
- **Di luar:** integrasi ke sistem pengadaan eksternal (SPSE, e-purchasing sungguhan; aplikasi hanya mencatat metode dan dokumennya), tanda tangan atau persetujuan digital, pembayaran sungguhan, sinkron kalender, notifikasi WhatsApp, aplikasi mobile, dan banyak instansi.

Asumsi

- Satu instansi, satu server, satu proses backend (bus live, cache, dan batas laju disimpan di memori).
- Tamu memiliki akun Google dengan email terverifikasi; zona waktu mengikuti Asia/Jakarta.
- Isi surat tidak diperiksa sistem: status surat otomatis dari adanya file PDF/JPG/PNG asli (maksimal 8 MB).
- Pajak kendaraan hanya diingatkan lewat dashboard (≤ 14 hari), bukan lewat email.
- SMTP (Gmail atau email kantor) tersedia. **Fitur email baru ditulis 5 Nov 2026 dan belum diuji dengan SMTP sungguhan.**
- Target N1 adalah asumsi latihan, bukan hasil ukur.

Bagian 2 · Catatan keputusan arsitektur

Tahap 2 · pilihan, alasan, konsekuensi, asumsi, dan bagian diagram yang terdampak.

K1 · Modular monolith longgar: satu proses Node.js (Express) dengan modul per fitur, bukan microservices	
Alasan	Satu instansi, tim kecil, dan satu database. Pembayaran dan booking butuh transaksi dan aturan yang konsisten; satu aplikasi lebih mudah dijalankan dan dipantau. Kode dipisah per fitur (Pemeliharaan, Pengadaan, Kendaraan, Ruang Rapat, Akun, Riwayat) di atas lapisan bersama: auth, audit, cache, dan live.
Konsekuensi	**Batas antarmodul belum dijaga ketat:** modul tamu memakai fungsi bantu dari Ruang Rapat, dan semua tabel berbagi satu skema serta menunjuk ke USERS. Kopling ini terlihat di C4 Level 3 (rute) dan Level 4. Semua modul di-deploy bersama. Backend wajib **satu proses** karena bus live, cache respons, dan batas laju disimpan di memori; menambah instance berarti memindahkan ketiganya ke penyimpanan bersama (mis. Redis).
Asumsi	Beban muat di satu server (lihat N1, belum diukur ulang).
Terdampak	Kotak **Backend API**; panah Web app → Backend (HTTPS/JSON + SSE). Terkait: N1.

K2 · Cegah booking ganda di server dan database, bukan di tampilan

Alasan	Tanpa kunci, dua request bersamaan sama-sama lolos cek bentrok. Jadwal di layar bisa basi, jadi cek di UI saja tidak cukup. Backend memakai transaksi dengan kunci per ruangan (`pg_advisory_xact_lock`) lalu mencari booking yang beririsan.
Konsekuensi	Request untuk ruangan yang sama diproses bergantian (antre singkat). **Tidak ada constraint database yang menjaga bentrok**, sehingga aturan ini hanya berlaku bila semua penulisan lewat backend; penulisan langsung lewat pgAdmin bisa melanggarnya.
Asumsi	Semua perubahan booking dilakukan lewat backend.
Terdampak	Panah Backend → Database; entity **RUANG_RAPAT**. Terkait: F5, N4.

K3 · Tamu masuk dengan Google OAuth 2.0 (PKCE); akun tamu terpisah dari akun pegawai; sesi 7 hari

Alasan	Tamu tidak perlu mendaftar atau mengingat kata sandi. Email terverifikasi Google dipakai juga untuk notifikasi. Akses tamu dibatasi pada booking miliknya lewat `guest_id`.
Konsekuensi	Bergantung pada Google: bila terganggu, tamu tidak bisa mengajukan (jadwal publik tetap terbaca). Bila sesi habis saat mengisi form, isian hilang karena tidak ada draf.
Asumsi	Semua tamu memiliki akun Google.
Terdampak	Kotak **Google (SSO)**; panah Backend → Google; entity **GUEST_ACCOUNTS**. Terkait: F4, F6, N2.

K4 · Email dikirim di latar belakang; pengingat memakai penjadwal 5 menit dan penanda di database

Alasan	SMTP yang lambat atau gagal tidak boleh menggagalkan booking. Penanda `reminder_h1_at` / `reminder_jam_at` diklaim lewat UPDATE bersyarat sehingga pengingat tidak terkirim dua kali.
Konsekuensi	Tidak ada antrean yang tahan restart: email yang gagal setelah 3 percobaan, atau terpotong restart backend, tidak dikirim ulang. Hitungan percobaan ada di memori.
Asumsi	Volume email rendah dan SMTP cukup andal. **Belum diuji dengan SMTP sungguhan.**
Terdampak	Panah Backend → Email; kolom `reminder_*` pada **RUANG_RAPAT**. Terkait: F7, N5.

K5 · Cache respons GET 5 detik yang dikosongkan setiap ada perubahan; SSE hanya mengirim sinyal

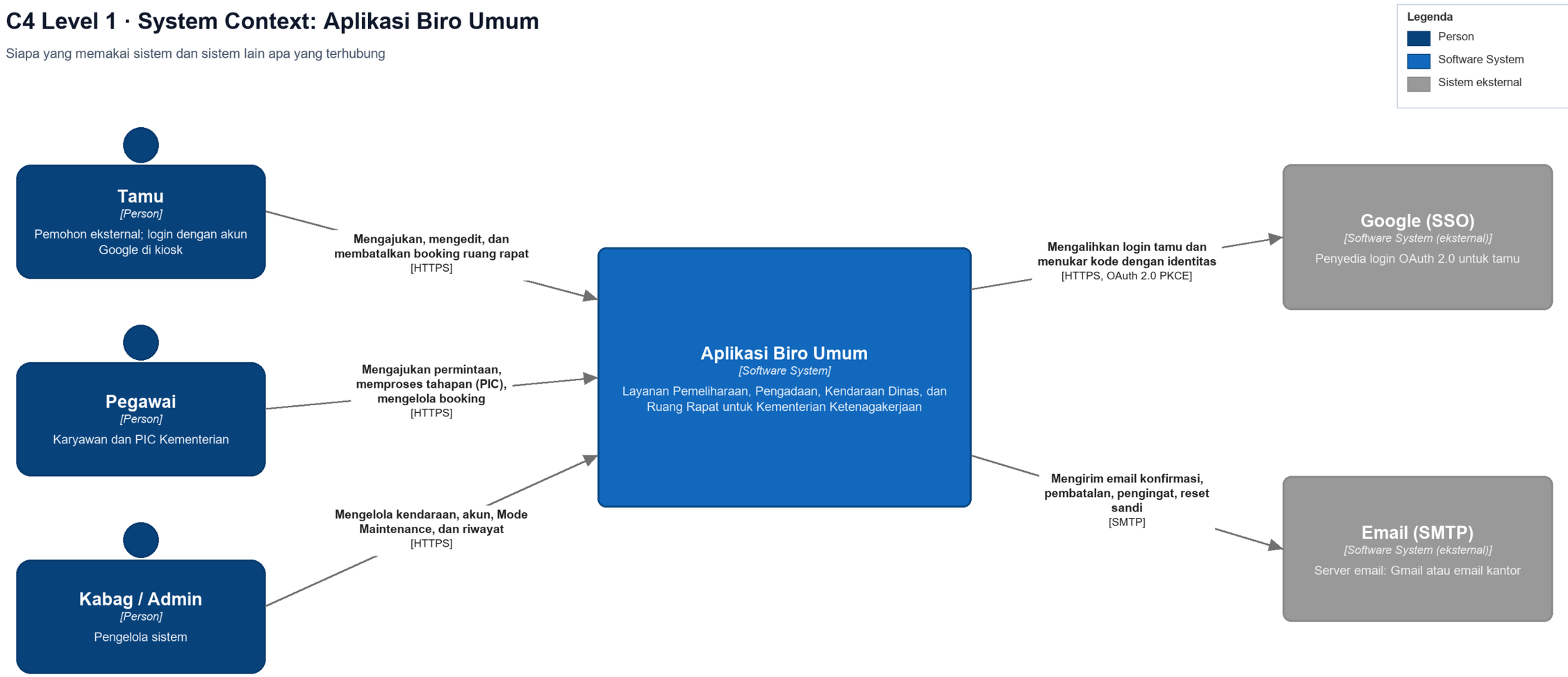
Alasan	Uji beban 4 Nov 2026 menunjukkan kode lama lag di sekitar 200 pengguna aktif: respons daftar berukuran besar dan dimuat ulang oleh banyak tab sekaligus.
Konsekuensi	Perubahan data langsung lewat pgAdmin bisa terlambat sampai 5 detik. Cache ada di memori satu proses (terkait K1).
Asumsi	Sinyal SSE cukup ringan untuk ribuan tab (belum diukur).
Terdampak	Panah Web app ↔ Backend (HTTPS/JSON + SSE). Terkait: N1.

K6 · Dokumen disimpan sebagai berkas di disk; salinan base64 di database tetap ada untuk data lama

Alasan	Berkas di folder `uploads/` lebih ringan dilayani daripada isi dokumen di database. Kolom `*_file_data` (base64) masih ada di skema dan dipakai sebagai cadangan bagi data lama yang belum punya berkas di disk.
Konsekuensi	Database lebih besar dan cadangannya ikut besar. Daftar harus mengecualikan kolom `*_file_data` agar respons tidak berukuran megabita. Dua tempat penyimpanan berarti bisa tidak sinkron. Pembersihan berkas yatim memerlukan skrip tersendiri.
Asumsi	Jumlah dokumen satu instansi masih wajar untuk satu disk lokal.
Terdampak	Kotak **Penyimpanan file**; panah Backend → file; kolom `*_file_path` dan `*_file_data`. Terkait: F1, F2, N1.

Bagian 3 · Diagram C4 · Level 1 · System Context

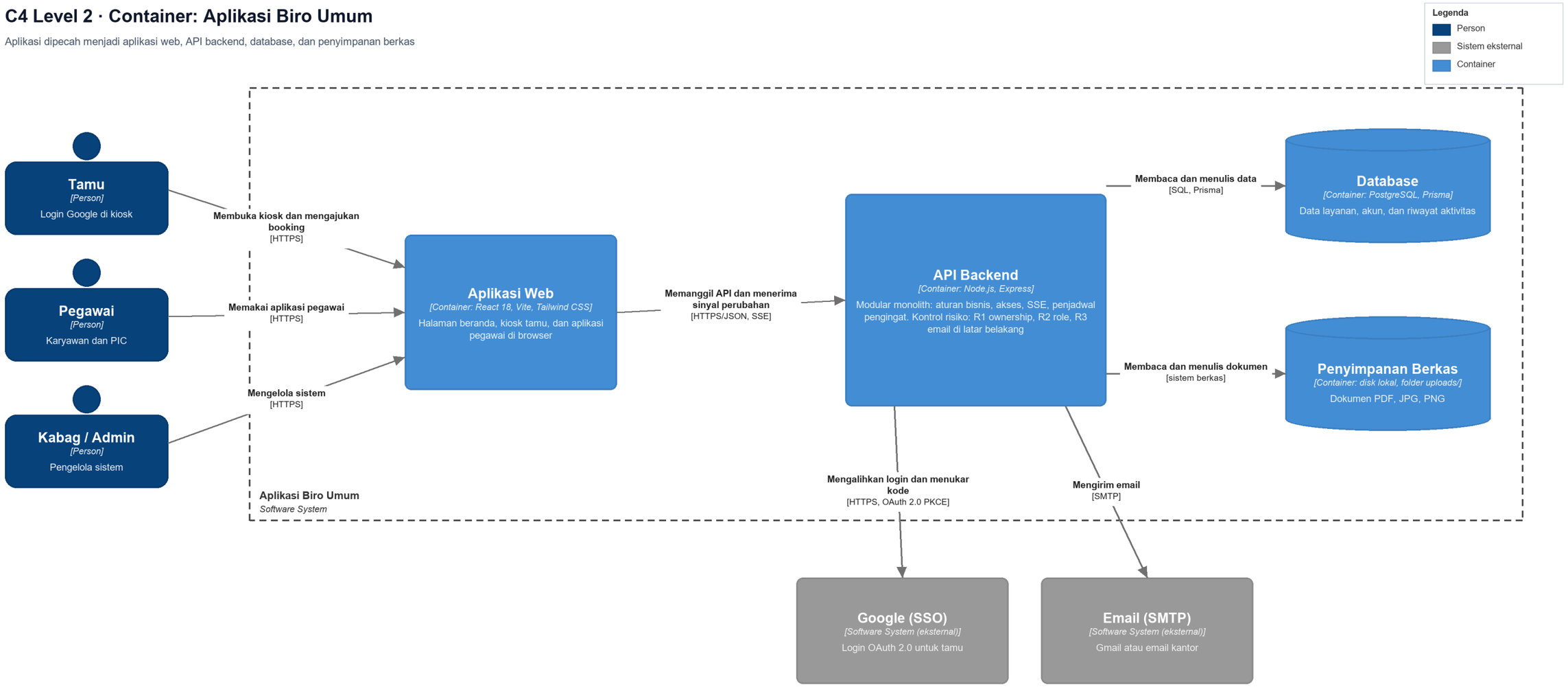
Siapa memakai sistem dan sistem lain apa yang terhubung.



- Tiga jenis pengguna: **Tamu** (login Google di kiosk), **Pegawai** (karyawan dan PIC), dan **Kabag/Admin**. Dua sistem luar: Google (login tamu) dan server email.
- Tidak ada integrasi ke sistem pengadaan eksternal; aplikasi hanya mencatat metode dan dokumennya.

Bagian 3 · Diagram C4 · Level 2 · Container

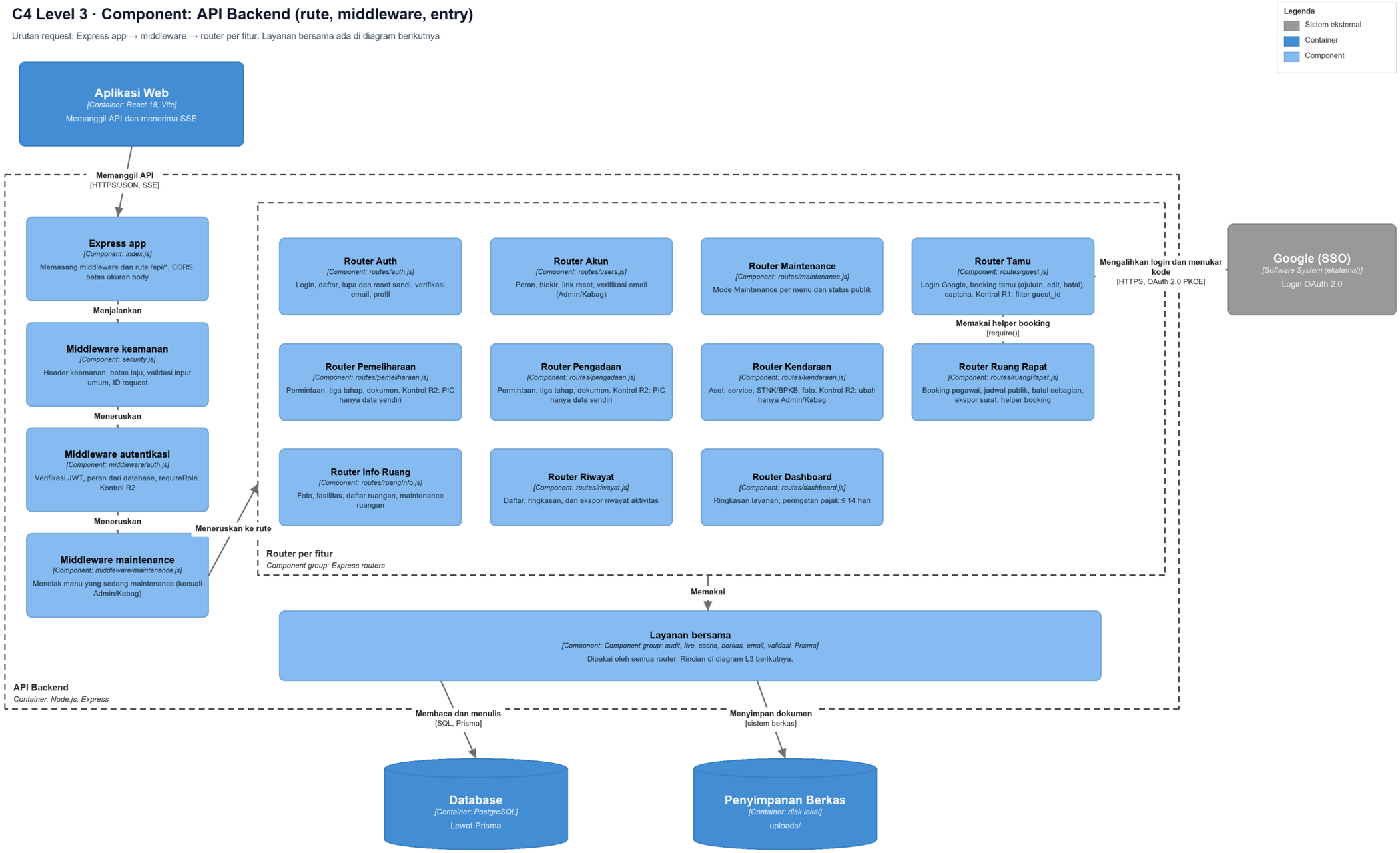
Aplikasi dipecah menjadi aplikasi web, API backend, database, dan penyimpanan berkas.



- Aplikasi web (SPA) dan API terpisah: HTTPS/JSON ditambah SSE yang hanya mengirim sinyal perubahan. Backend berjalan sebagai **satu proses**; dokumen disimpan sebagai berkas di `uploads/`.
- **Kontrol risiko** berjalan di kotak **API Backend**: R1 cek ownership, R2 cek role dan pemilik data, R3 email di latar belakang (rincian di bagian 5). Contoh penelusuran fitur (Tahap 5): F4 melewati Aplikasi Web → API Backend → Database (entity RUANG_RAPAT) dan, bila ada surat, penyimpanan berkas; F1 melewati Aplikasi Web → API Backend → Database (PEMELIHARAAN atau PENGADAAN).

Bagian 3 · Diagram C4 · Level 3 · Component: API Backend (rute)

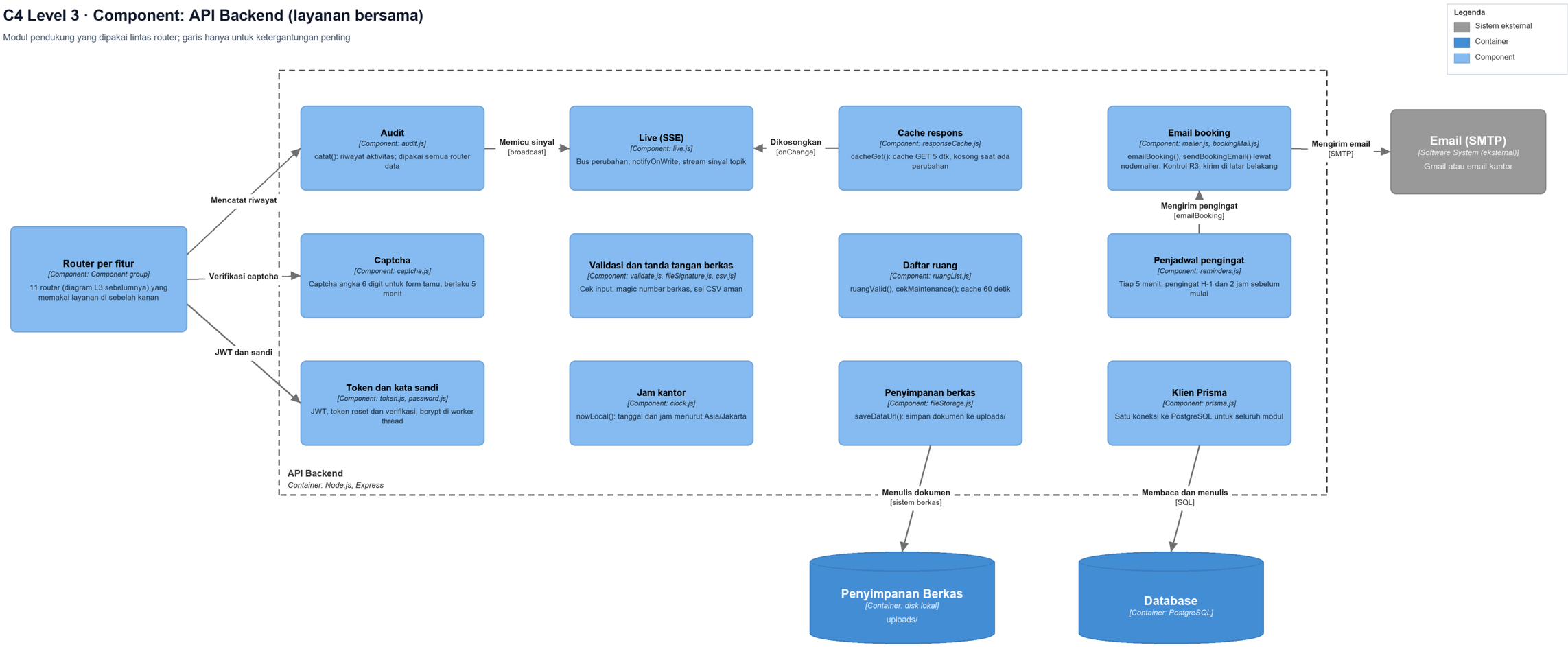
Alur request: Express app → middleware keamanan, autentikasi, maintenance → router per fitur.



- **Router Tamu memakai helper dari Router Ruang Rapat** (panah “Memakai helper booking”): kopling antarmodul yang belum dibatasi antarmuka (K1). Hanya Router Tamu yang berbicara ke Google.
- Penanda **R1** (Router Tamu) dan **R2** (Middleware autentikasi, Router Pemeliharaan, Pengadaan, Kendaraan) menunjukkan tempat kontrol keamanan berjalan.

Bagian 3 · Diagram C4 · Level 3 · Component: API Backend (layanan bersama)

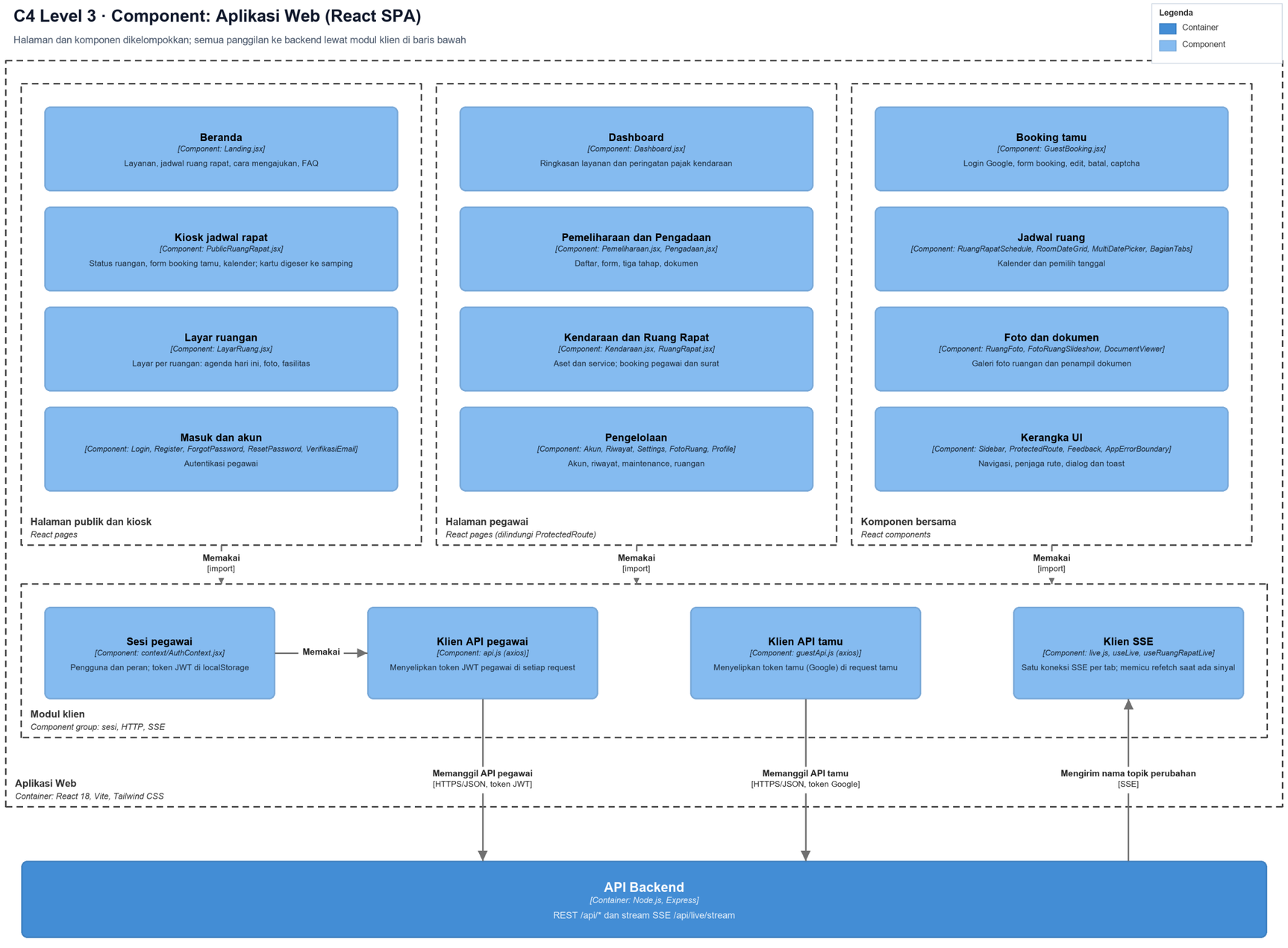
Modul pendukung yang dipakai lintas router; garis hanya untuk ketergantungan penting.



- **Audit** memicu sinyal live dan **cache respons** dikosongkan lewat bus yang sama, jadi data tidak basi setelah ada perubahan. **Email booking** memuat kontrol **R3** (kirim di latar belakang).
- **Penjadwal pengingat** memakai `emailBooking` untuk mengirim email, dan membaca serta menulis booking lewat Prisma (garisnya tidak digambar agar terbaca).

Bagian 3 · Diagram C4 · Level 3 · Component: Aplikasi Web (React SPA)

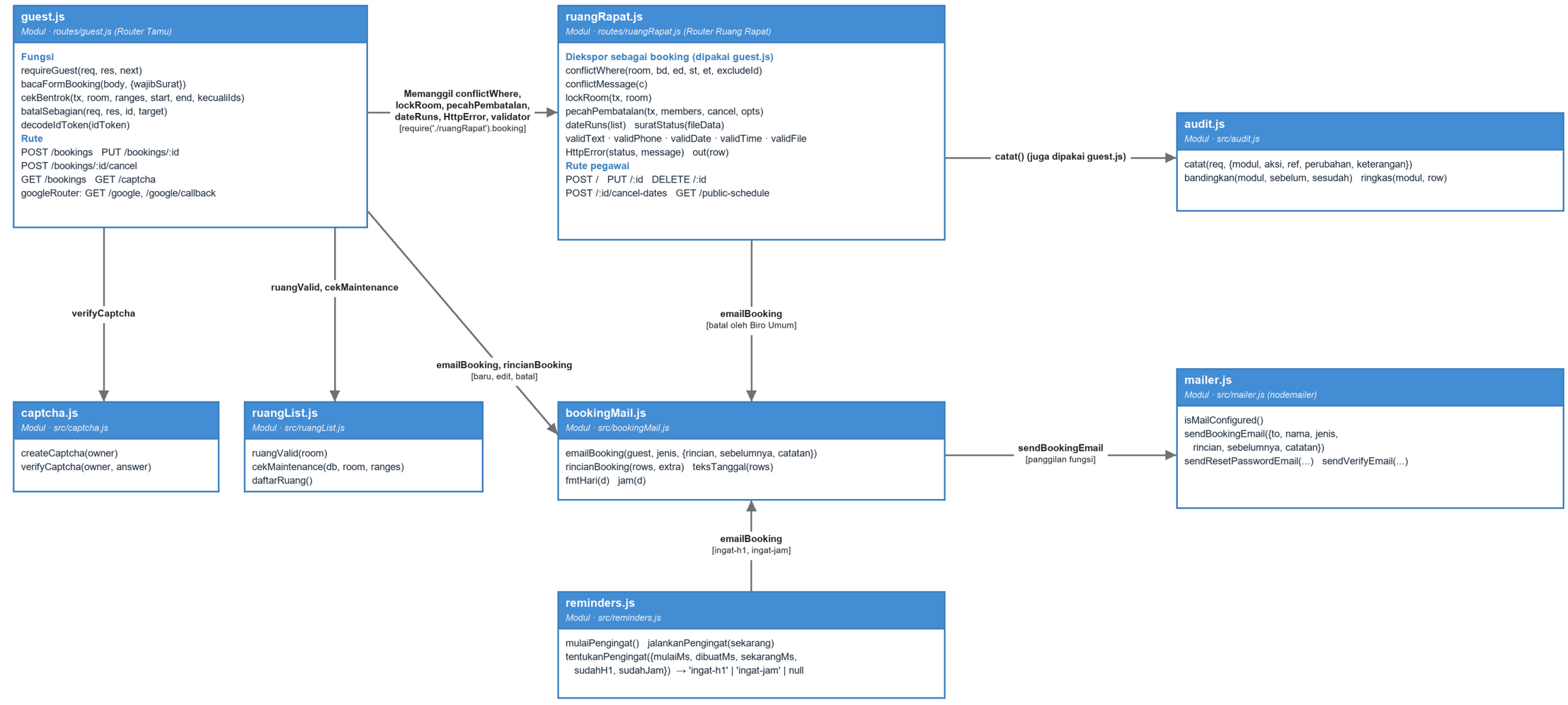
Halaman dan komponen dikelompokkan; semua panggilan ke backend lewat modul klien.



- Dua klien HTTP terpisah: **api.js** membawa token pegawai (JWT), **guestApi.js** membawa token tamu (Google). Satu koneksi SSE per tab memicu halaman memuat ulang datanya.
- Kiosk (**PublicRuangRapat**) menampilkan kartu status ruangan; bila “Semua bagian” dipilih, kartu digeser ke samping.

Bagian 3 · Diagram C4 · Level 4 · Code: modul booking ruang rapat

Fungsi utama dan siapa memanggil siapa pada alur ajukan, edit, batal, dan pengingat booking.



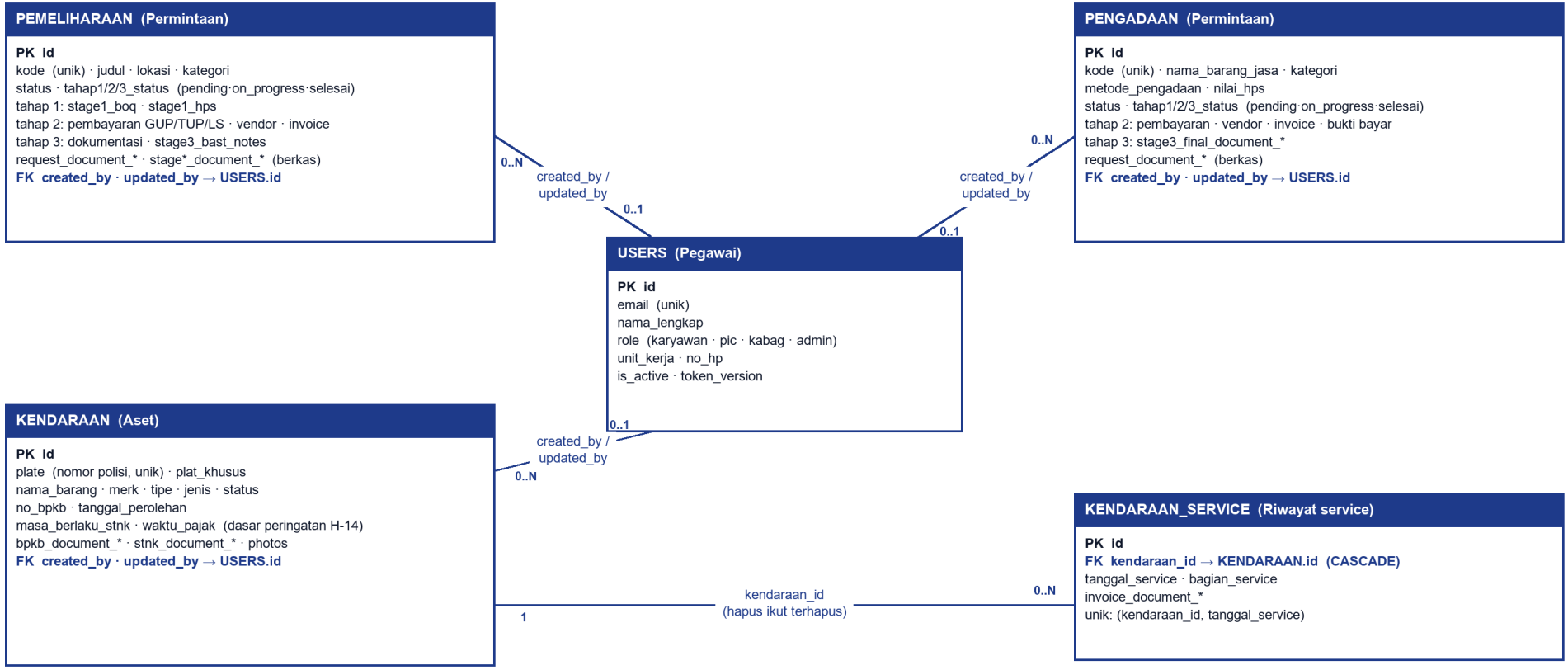
- `guest.js` memanggil `conflictWhere`, `lockRoom`, dan `pecahPembatalan` milik `ruangRapat.js` lewat ekspor `booking`; ini inti pencegahan booking ganda (K2).
- Email booking melewati satu pintu: `emailBooking` di `bookingMail.js` → `sendBookingEmail` di `mailer.js`. Pemanggilnya: router tamu, router pegawai (batal oleh Biro Umum), dan penjadwal pengingat.

Bagian 4 · Data (ERD 1): Pemeliharaan, Pengadaan, Kendaraan

Tahap 3 · garis tunggal = foreign key; semua FK ke USERS boleh kosong.

ERD 1: Pemeliharaan, Pengadaan, dan Kendaraan Dinas

PK = kunci utama, FK = foreign key. Ketiga layanan dicatat oleh pegawai (USERS); ERD 2 membahas Ruang Rapat



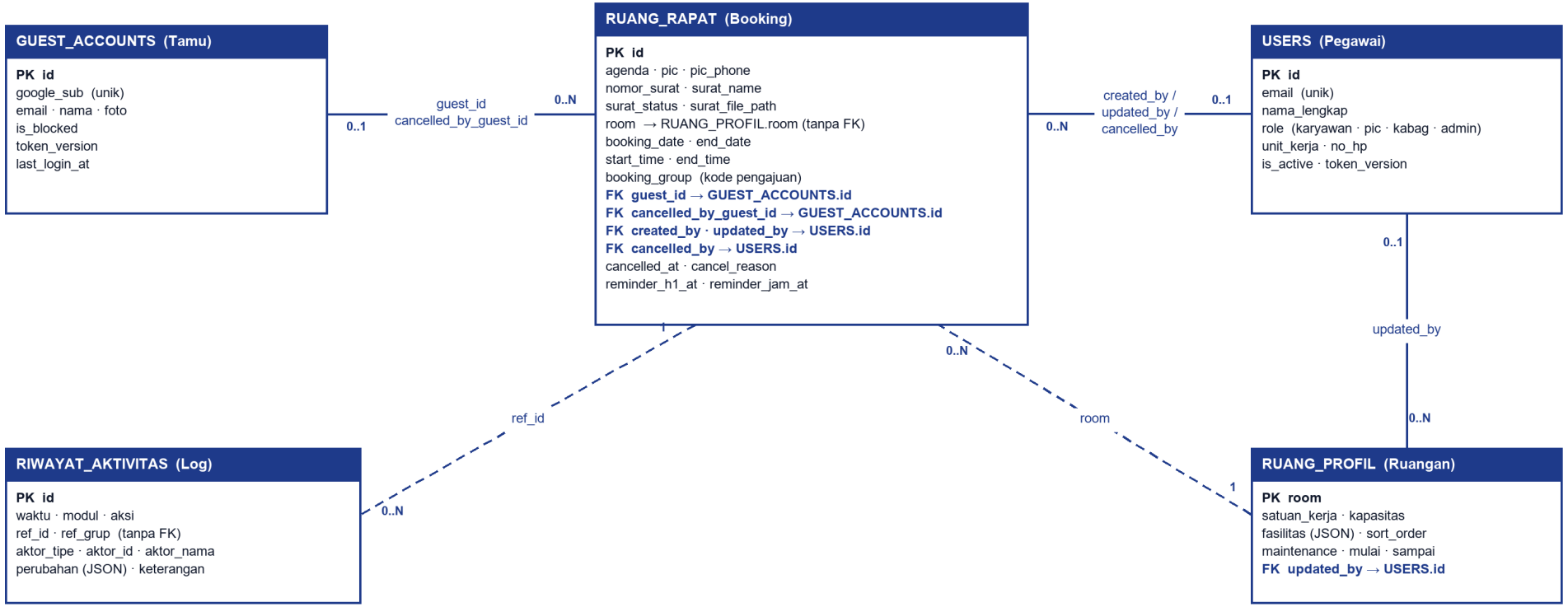
Pemeliharaan dan Pengadaan memakai pola yang sama: tiga tahap berstatus, dokumen per tahap. FK created_by/updated_by boleh kosong (ON DELETE SET NULL).
Tidak digambar agar terbaca: MAINTENANCE_MODE (status maintenance per menu) dan RUANG_FOTO (foto ruangan).

Bagian 4 · Data (ERD 2): Ruang Rapat

Tahap 3 · garis putus-putus = rujukan tanpa foreign key (dijaga aplikasi).

ERD 2: Data Booking Ruang Rapat

PK = kunci utama, FK = foreign key. Garis putus-putus = rujukan tanpa FK (dijaga aplikasi)



Satu pengajuan = satu booking_group = satu atau lebih baris RUANG_RAPAT (rentang tanggal berurutan; tanggal loncat-loncat = beberapa baris).
Booking tidak dihapus saat batal: cancelled_at + alasan diisi. Semua FK boleh kosong dan memakai ON DELETE SET NULL.

Bagian 4 · Aturan data dan integrasi

Tahap 3 · setiap relasi dan panah bisa dijelaskan.

Aturan data: Pemeliharaan, Pengadaan, Kendaraan

- **Pemeliharaan dan Pengadaan** punya `kode` unik, status keseluruhan, dan tiga status tahap (`pending`, `on_progress`, `selesai`). Dokumen per tahap disimpan sebagai berkas (K6). Metode pembayaran Pemeliharaan: GUP, TUP, atau LS.
- **Kendaraan:** nomor polisi (`plate`) unik; satu kendaraan hanya boleh punya satu catatan service per tanggal; menghapus kendaraan ikut menghapus riwayat servicenya (CASCADE). `waktu_pajak` menjadi dasar peringatan di dashboard.
- **Semua FK ke USERS boleh kosong dengan ON DELETE SET NULL:** menghapus akun pegawai tidak menghapus datanya.

Aturan data: Ruang Rapat

- **Satu pengajuan = satu** `booking_group` **= satu atau lebih baris** RUANG_RAPAT (rentang tanggal berurutan; tanggal loncat-loncat menjadi beberapa baris).
- **Bentrok** = booking aktif (`cancelled_at` kosong) di ruangan yang sama dengan tanggal **dan** jam yang beririsan. Dijaga backend lewat transaksi dan kunci ruangan (K2), bukan constraint database.
- **Batal tidak menghapus baris:** `cancelled_at`, `cancel_reason`, dan pembatal (pegawai: `cancelled_by`; tamu: `cancelled_by_guest_id`) diisi. Batal sebagian memecah rentang tanggal menjadi beberapa baris.
- **CHECK di database:** `end_time > start_time`, `end_date ≥ booking_date`, dan `surat_status` hanya belum / ditinjau / diterima.
- `room` **bukan FK** ke RUANG_PROFIL: kode ruangan dicek aplikasi saat menyimpan. Konsekuensinya, database sendiri tidak menolak nama ruangan sembarang bila ditulis langsung.
- **RIWAYAT_AKTIVITAS tanpa FK:** menyimpan sendiri nama, email, dan role aktor serta rujukan (`ref_id`, `ref_grup`); hanya Kabag/Admin yang bisa membacanya.

Integrasi

Integrasi	Dari → ke	Data	Kapan	Jika gagal
Google OAuth 2.0 (PKCE)	Browser → Google; Backend → Google (token)	Kode otorisasi; token ID berisi email, nama, foto, `sub`.	Tamu klik “Login Google”.	Tamu dialihkan balik dengan `guest_error=` dan bisa mencoba lagi.
Email (SMTP)	Backend → server SMTP	Penerima (email tamu atau pegawai), jenis email, dan rincian: agenda, ruangan, tanggal, jam (booking) atau tautan (reset/verifikasi).	Setelah ajukan, edit, batal; pengingat tiap 5 menit; daftar dan lupa kata sandi.	Dikirim di latar belakang: booking tetap tersimpan, kegagalan dicatat di log, pengingat dicoba ulang maks. 3 kali.
PostgreSQL	Backend → Database (Prisma)	Seluruh data aplikasi.	Setiap request.	Respons 500 dengan pesan umum; detail di log. Penulisan penting memakai transaksi.
SSE	Backend → browser	Sinyal “ada perubahan” saja, bukan datanya.	Setiap perubahan data berhasil.	Browser tersambung ulang otomatis; cache server dikosongkan saat ada perubahan.
Penyimpanan file	Backend → disk (`uploads/`)	Dokumen PDF/JPG/PNG (surat, permohonan, invoice, BAST, STNK/BPKB, foto).	Saat data menyertakan dokumen.	Bila berkas hilang, dibaca dari salinan base64 di database (data lama).

Bagian 5 · Akses dan risiko

Tahap 4 · siapa boleh melakukan apa. Baris = peran, kolom = tindakan.

Peran \ Tindakan	Lihat jadwal ruang rapat	Buat booking ruang rapat	Edit / batal booking ruang rapat	Ajukan Pemeliharaan / Pengadaan	Edit / hapus Pemeliharaan / Pengadaan	Lihat data kendaraan	Tambah / ubah / hapus kendaraan	Kelola akun, Maintenance, Riwayat
Tamu (login Google)	Boleh	Boleh	Hanya miliknya	Tidak	Tidak	Tidak	Tidak	Tidak
Karyawan	Boleh	Boleh	Boleh	Boleh	Tidak	Boleh	Tidak	Tidak
PIC	Boleh	Boleh	Boleh	Boleh	Hanya miliknya	Boleh	Tidak	Tidak
Kabag	Boleh	Boleh	Boleh	Boleh	Boleh	Boleh	Boleh	Boleh
Admin	Boleh	Boleh	Boleh	Boleh	Boleh	Boleh	Boleh	Boleh

- **Tamu** masuk lewat login Google di kiosk dan tidak bisa membuka aplikasi internal. “Hanya miliknya” = pengajuan atas akun Googlenya sendiri, selama belum berjalan.
- **Ruang rapat:** semua pegawai sengaja boleh mengelola semua booking, termasuk surat dan statusnya.
- **PIC** adalah karyawan yang otomatis naik peran setelah mengajukan Pemeliharaan/Pengadaan pertamanya. “Hanya miliknya” = data yang ia tambahkan sendiri.
- **Kolom terakhir** mencakup Akun & Akses, Mode Maintenance, Riwayat Aktivitas, serta foto dan daftar ruangan. Kabag setara Admin, kecuali akun **Admin** tidak bisa diubah Kabag.
- Pembatasan diterapkan di **server** (kotak API Backend), bukan hanya dengan menyembunyikan tombol di tampilan.

Bagian 5 · Catatan risiko

Tahap 4 · dua risiko keamanan dan satu risiko sistem gagal; tiap risiko memuat kejadian, dampak, tindakan, cara uji, dan kotak tempat kontrol berjalan.

R1 · Keamanan: tamu mengakses booking milik tamu lain (IDOR)	
Kejadian	ID booking berupa angka berurutan. Tamu yang sudah login mengganti ID pada `PUT /api/guest/bookings/:id` atau `POST /api/guest/bookings/:id/cancel` dengan ID milik tamu lain lalu mengubah atau membatalkannya.
Dampak	Booking orang lain berubah atau batal tanpa izin; ruangan bisa dikosongkan atau diambil; tamu yang dirugikan tidak tahu penyebabnya.
Tindakan	Cek ownership: setiap query selalu menyertakan `guest_id` tamu yang login (bukan hanya ID booking). Booking orang lain dijawab **404**, bukan 403, agar ID yang valid tidak bisa ditebak. Dilakukan di server.
Cara uji	Login sebagai tamu A dan B; buat booking dengan B dan catat ID-nya. Dengan token A panggil PUT edit dan POST cancel ke ID itu. **Benar bila** keduanya 404 dan booking B tidak berubah.
Kotak kontrol	**API Backend** → **Router Tamu** (filter `guest_id`; penanda R1 di C4 Level 3). Kebutuhan: F6, N2.

R2 · Keamanan: pegawai memanggil API di luar wewenang perannya (akses lintas peran)	
Kejadian	Karyawan atau PIC tidak melihat tombol Edit/Hapus, tetapi memanggil API langsung (DevTools atau Postman), mis. `PUT /api/pemeliharaan/:id` milik PIC lain atau `POST /api/kendaraan`.
Dampak	Data pemeliharaan, pengadaan, atau kendaraan orang lain bisa diubah atau dihapus. Riwayat mencatat pelakunya, tetapi kerusakan data sudah terjadi.
Tindakan	Batasi role: `requireRole` di server (Pemeliharaan/Pengadaan: kabag, pic, admin; Kendaraan dan Akun & Akses: admin, kabag) ditambah cek pemilik: PIC hanya boleh bila `created_by` sama dengan ID penggunanya. Peran dibaca dari database tiap request.
Cara uji	Token **karyawan**: PUT dan DELETE ke data Pemeliharaan, lalu POST ke Kendaraan. Token **PIC B**: PUT ke data buatan PIC A. **Benar bila** semuanya 403 dan data tidak berubah; ulangi dengan token Kabag untuk memastikan yang berhak tetap lolos.
Kotak kontrol	**API Backend** → **Middleware autentikasi** dan router Pemeliharaan, Pengadaan, Kendaraan (penanda R2 di C4 Level 3). Kebutuhan: F2, N3.

R3 · Sistem gagal: SMTP bermasalah, email konfirmasi dan pengingat tidak terkirim	
Kejadian	Server email menolak login, kena batas kirim, atau tidak bisa dihubungi. Email konfirmasi, pembatalan oleh Biro Umum, dan pengingat tidak sampai ke tamu.
Dampak	Tamu tidak tahu bookingnya dibatalkan dan datang sia-sia, atau lupa jadwal sehingga ruangan terpakai kosong. Bila pengiriman email ikut menggagalkan simpan booking, tamu tidak bisa memesan sama sekali.
Tindakan	Kirim di latar belakang: kegagalan hanya dicatat di log dan tidak menggagalkan simpan atau batal. Pengingat yang gagal dilepas klaimnya dan dicoba lagi sampai 3 kali. Sumber kebenaran tetap kartu “Booking Anda” di kiosk. Keterbatasan: hitungan percobaan hilang bila backend restart.
Cara uji	Isi `SMTP_PASS` dengan nilai salah, restart backend, lalu ajukan, edit, dan batalkan booking. **Benar bila** semuanya tetap berhasil dan log memuat “Email booking tamu gagal dikirim”. Kosongkan `SMTP_HOST` untuk memastikan fitur email diam tanpa error. **Belum dijalankan:** kode email baru ditulis dan belum diuji.
Kotak kontrol	**API Backend** → **Email booking** (penanda R3 di C4 Level 3) dan panah ke **Email (SMTP)**. Kebutuhan: F7, N5.